C Programming Tutorial

OPERATING SYSTEMS

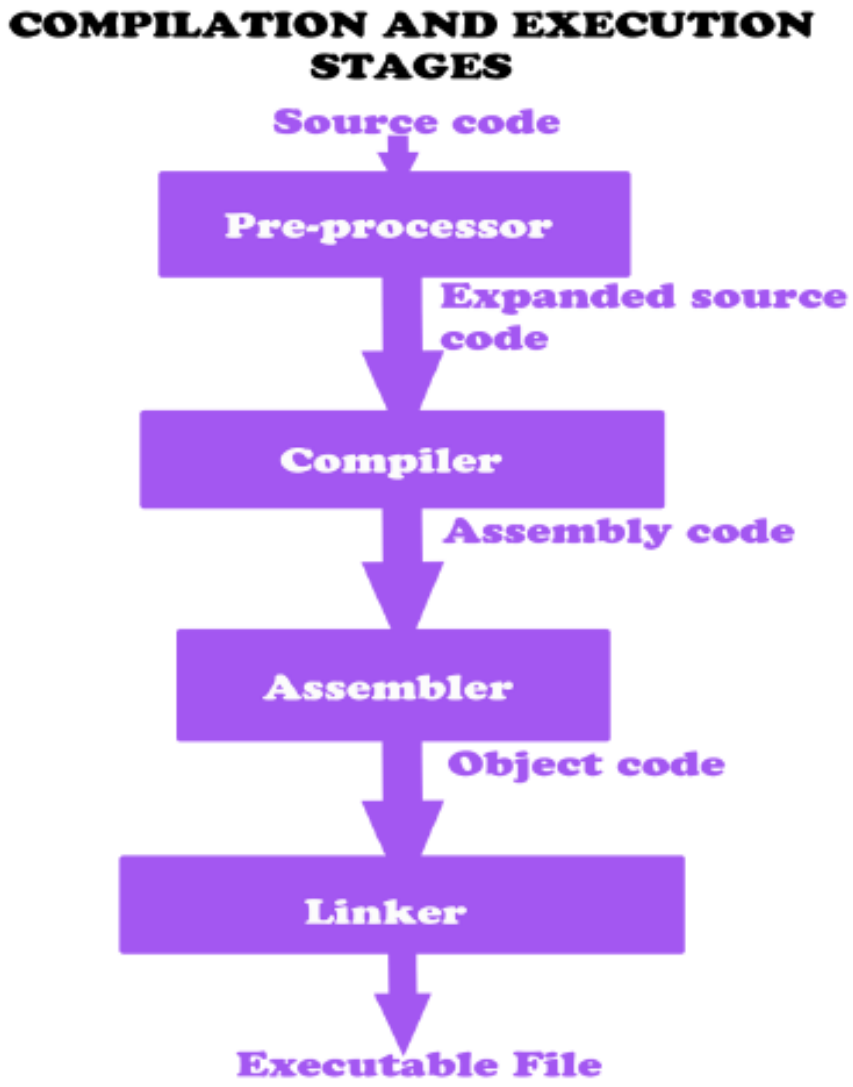
Stages of a C program

1. Editor
    1. Programmer creates program in an editor and stores it on disk
2. Preprocessor
    1. Macro substitution
    2. Stripping of comments
    3. Expansion of included files
3. Compiler
    1. Takes the output of the preprocessor, the source code, and generates assembler source code
4. Assembler
    1. Takes the file produced by the assembler and generates an object file which contains machine level instructions

Stages of a C program

5. Linker
    1. Links function calls with their definitions (links object code with the libraries)
    2. Adds extra code for the program to start and end
    3. Produces the executable file and stores it on disk
6. Loader
    1. Puts the program in memory to be ready for execution by the CPU

Check this link for more details about each stage:
https://www.calleerlandsson.com/posts/the-four-stages-of-compiling-a-c-program/



Stages of a C program

Pieces of C

1. Types and Variables
    1. Primitive Types: char, float, int, and double
    2. Pointers and Arrays
    3. Structures
2. Expressions
    1. Arithmetic, logical, and assignment operators
3. Statements
    1. Conditional, iteration, and branching instructions
4. Functions
    1. Group of statements and variables

Variables in C

- Integer: int %d
- Floating point number: float %f
- Characters: char %c
- Double: double
- Absence of a type: void

You can declare variables simultaneously if they are of the same type:
- int a, b, c;

Variables can be initialized (assigned an initial value) in their declaration. The initializer consists of an equal sign followed by a constant expression:
- type *var_name* = value;

Always initialize your variables!

Variables in C

1. **Characters Allowed :**
   - Underscore(_)
   - Capital Letters (A –Z)
   - Small Letters (a –z)
   - Digits (0 –9)
2. **Blanks & Commas** are not allowed
3. No Special Symbols other than **underscore(_) are allowed**
4. **First Character** should be **alphabet or Underscore**
5. Variable name Should not be **Reserved Word**

Reserved words

auto	**double**	**int**	**struct**
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Arithmetic operators

Operator	Description
+	Adds two operands.
–	Subtracts second operand from the first.
*	Multiplies both operands.
/	Divides numerator by de-numerator.
%	Modulus Operator and remainder of after an integer division.
++	Increment operator increases the integer value by one.
--	Decrement operator decreases the integer value by one.

Logical operators

Operator	Description
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.
||	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.

Relational operators

Operator	Description
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the condition becomes true.

Assignment operators

- =
- += e.g. x += y → x = x + y
- -= e.g. x -=y → x = x - y
- *= e.g. x *=y → x = x * y
- /=
- %=

Escape codes

Similar to Java:

- \n new line
- \t tab
- \\ backslash
- \” double quote

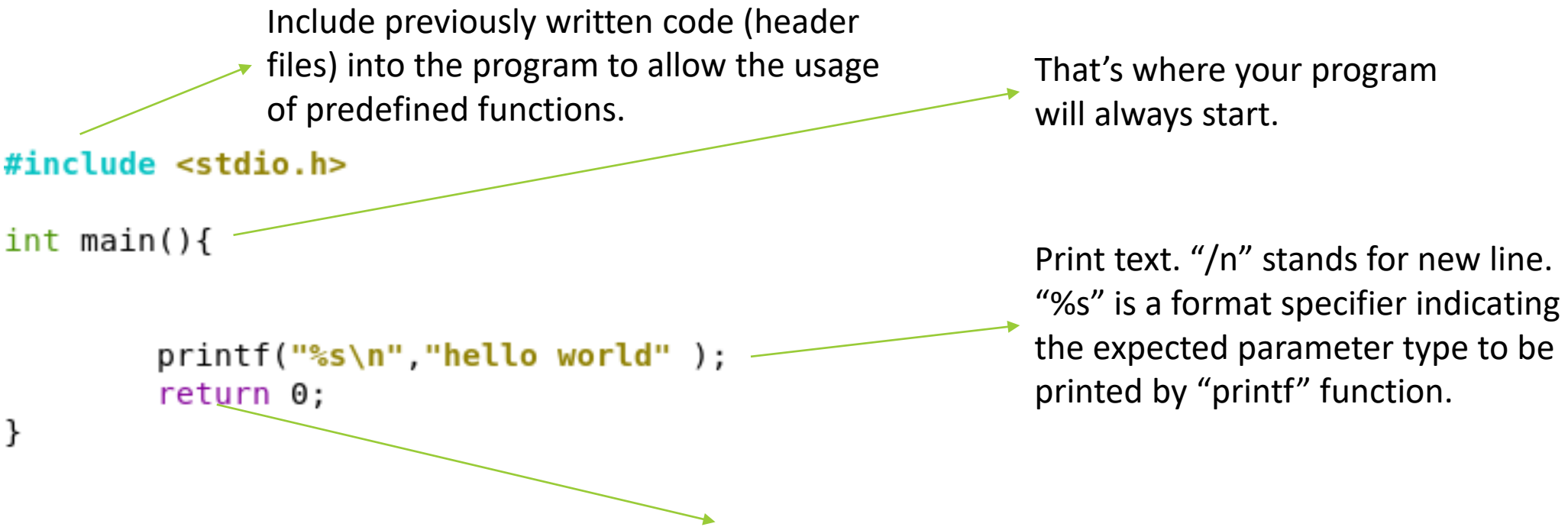
Compile and run C

1. Run the following command:
   - **gcc** -o hello hello.c *OR* gcc –o hello.out hello.c
   - The -o option specifies the name to be assigned to the compiled/executable program.
2. Run the following on the terminal:
   - ./hello *OR* ./hello.out

GCC(GNU Compiler Collection): A compiler.

C Syntax

Lets start learning the C Syntax via the famous Hello World example.



C Syntax

In the previous slide we encountered the first C line of code. We will discuss this line of code more in this slide.

```
#include <stdio.h>
```

Any line that starts with a “#” is a command. As such, “#include <stdio.h>” is a command.

Basically, we are including previous code into our program. “stdio.h” is a header file that defines the “printf” function and other functions.

“stdio.h” stands for standard input output header.

The usage of <> symbol indicates that the header file is located in the standard search path otherwise , “” symbol can be used instead to look first in the current directory.

C Syntax

Lets take another example.

```c
#include <stdio.h>


int main(){
        int a = 2;
        int b = 3;

        printf("%d\n",a+b);
        return 0;
}
```

Declare integer variable a and assign 2 to its value.

Declare integer variable b and assign 3 to its value.

Print the value of a + b as a decimal. “%d” is a format specifier for “printf” indicating the expected parameter type.

Exercise

- Write a C program that converts length from meter to kilometer.

  Hardcode the input length for now (don't worry about reading from a user)
  e.g. input = 4500.5

Remember

gcc -o hello hello.c *OR* gcc –o hello.out hello.c
./hello *OR* ./hello.out

Note: gcc and -o are separate
Note: ./ and hello are connected

Exercise - Answer

```c
#include <stdio.h>

int main(){

        double input =  4500.5;

        double kilometer = input/1000;

        printf("length in kilometer is %f\n",kilometer);

        return 0;
}
```

C Syntax

Lets look at a third example.

```c
#include <stdio.h>
int sum(int, int);

int main(){
        printf("%d\n",sum(2,3));
        return 0;
}

int sum(int a ,int b){
        return a+b;
}
```

Function's prototype or declaration. A body is not needed and it just has an interface.

Call the function "sum" with 2 arguments and print its result.

Function's definition. Actual logic is implemented within the function.

Exercise

- Write a C program that contains two functions:
  - The first function converts the input length from meter to kilometer
  - The second function converts the input temperature from Fahrenheit to Celsius:

$$T(c) = (T(f) - 32) * 5/9$$

  - Inside the main call these two methods

Hardcode the inputs for now.

To make sure the Fahrenheit conversion is working write check this site.

Exercise - Answer

```c
#include <stdio.h>
double lengthConverter(double);
double tempratureConverter(double);
int main(){

        double input =  4500;

        double temp = 35;

        printf("%f m = %f km \n",input,lengthConverter(input));
        printf("%f Fahrenheit is %f degrees Celsius\n",temp,tempratureConverter(temp));
        return 0;
}
double lengthConverter(double a){
        //convert meter to kilometer
        return a/1000;
}
double tempratureConverter(double temp){
        //convert Fahrenheit to Celsius
        return (temp-32)*5/9;
}
```